



灵感画布

LingoDraw

把 AIGC 创作流程变成可视化节点编排



项目概览

PROJECT OVERVIEW

把复杂的 **AIGC** 创作流程，
组织成 **可理解、可编排** 的画布体验。

01 项目定位

AIGC节点式画布工具

02 我的角色

UX/UI设计负责人

03 展示重点

体验架构 / 关键交互 / 设计系统 / 可交互Demo

设计推进

DESIGN OWNERSHIP

我负责的不是把界面画出来，
而是把复杂需求推进到可验证、可交付。

● 判断 / 验证 / 交付 ●

01 ● 建立判断

从业务目标与节点关系中
拆出核心体验问题。

02 ● 快速验证

用 Skill 与可交互 Demo
让关键行为先被操作。

03 ● 推进交付

沉淀高保真、交互规范、
设计系统与开发说明。

核心挑战

CORE CHALLENGE

难点不是功能多，
而是关系能否被理解和控制。

当文本、图片、视频、模型与参数进入同一张画布，
体验必须先回答三个问题。



设计目标

DESIGN CRITERIA

我把设计目标，
写成三条可以被验证的标准。

后续方案不靠“看起来更完整”，
而看它是否真正降低理解与操作成本。

01 无需预学

用户无需预先理解模型与全部参数，
也能建立第一条生成链路。

通过标准：入口先于配置，复杂能力按任务逐步出现。

02 关系可解释

用户能够说明输入、配置与结果之间
为什么这样连接。

通过标准：节点职责、依赖与流向在画布中持续可见。

03 状态可行动

用户在任一生成阶段，都能判断
当前状态与下一步。

通过标准：触发、等待、完成与结果承接均有明确反馈。

体验架构

EXPERIENCE ARCHITECTURE

画布不规定一种起点，而是让不同输入
按任务组合成可继续编辑的创作路径。

文本可以单独开始，也可以与图片或视频共同建立上下文；
输出目标再决定进入图片配置还是视频配置。

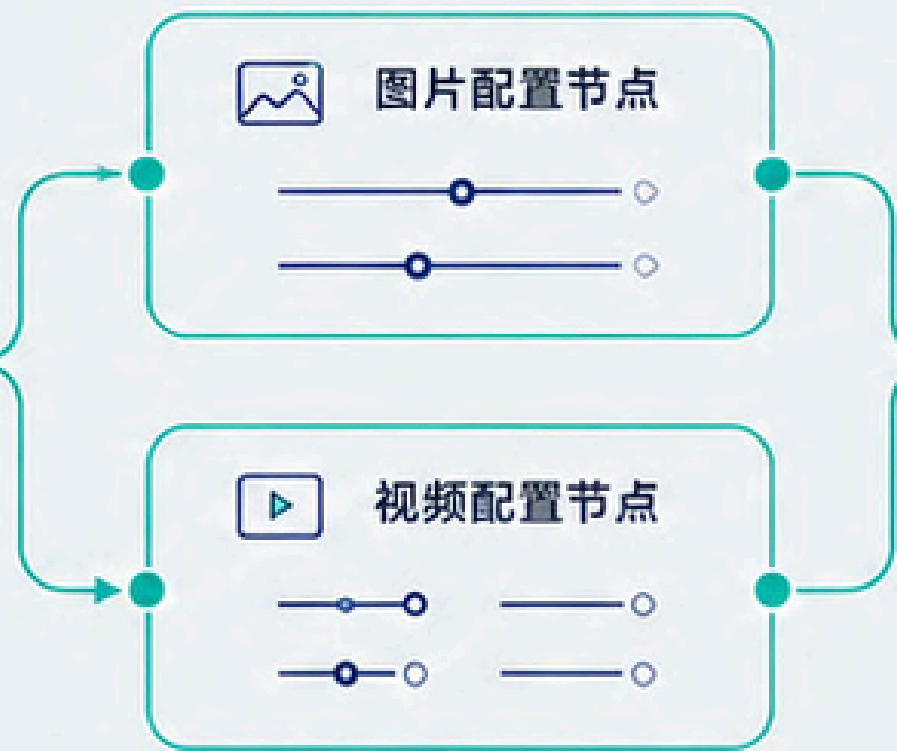
01 输入组合

文本可以单独开始，也可以与图片节点或视频节点共同建立生成上下文。



02 配置分流

根据输出目标，连接图片配置节点或视频配置节点。



03 参数确认

模型、比例、画质与数量，在对应配置节点中完成。



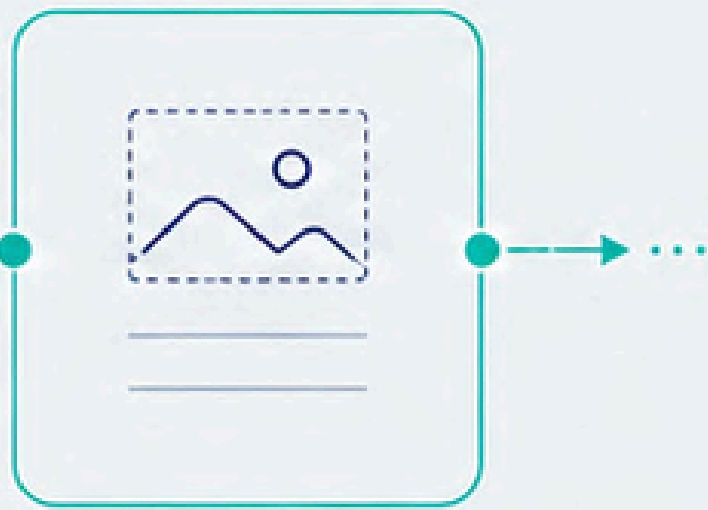
04 触发生成

参数确认后点击「立即生成」，任务沿当前连接进入生成。



05 结果续接

右侧建立结果节点，生成内容继续参与后续画布编排。



早期探索

EXPLORATION

先让用户迈出第一步，
再验证链路是否清楚。

低保真阶段，我先看空节点能否快速开始，
再看素材角色、参数选择和生成入口是否能被理解。

02 角色可读

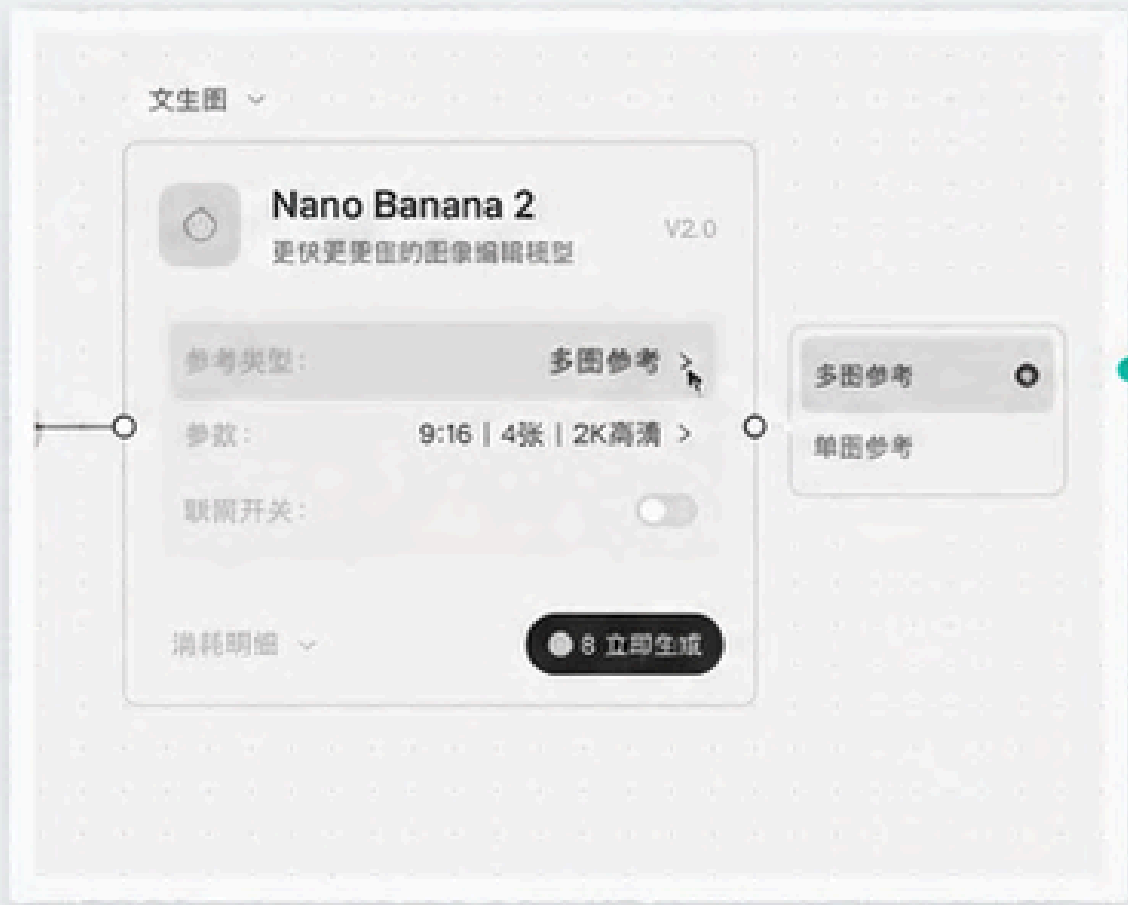
连线说明流向，语义标签
说明文本、主体图、参考图
分别承担什么角色。

01 快速开始

初始节点直接提供常用
任务按钮，降低空白画布
的学习成本和思考时间。

03 参数就地

参数在配置节点内完成
选择，确认后再进入生成，
不让用户跳出当前链路。



原型协作

PROTOTYPE

我先跑通关键行为，再用 Demo 对齐团队判断。

我把业务目标、节点规则和状态边界写进 Skill，
用可操作 Demo 验证关键行为；确认后，
再进入高保真与开发说明。

- 01

写成规则

把节点职责、操作约束和状态边界整理成可执行 Skill。
- 02

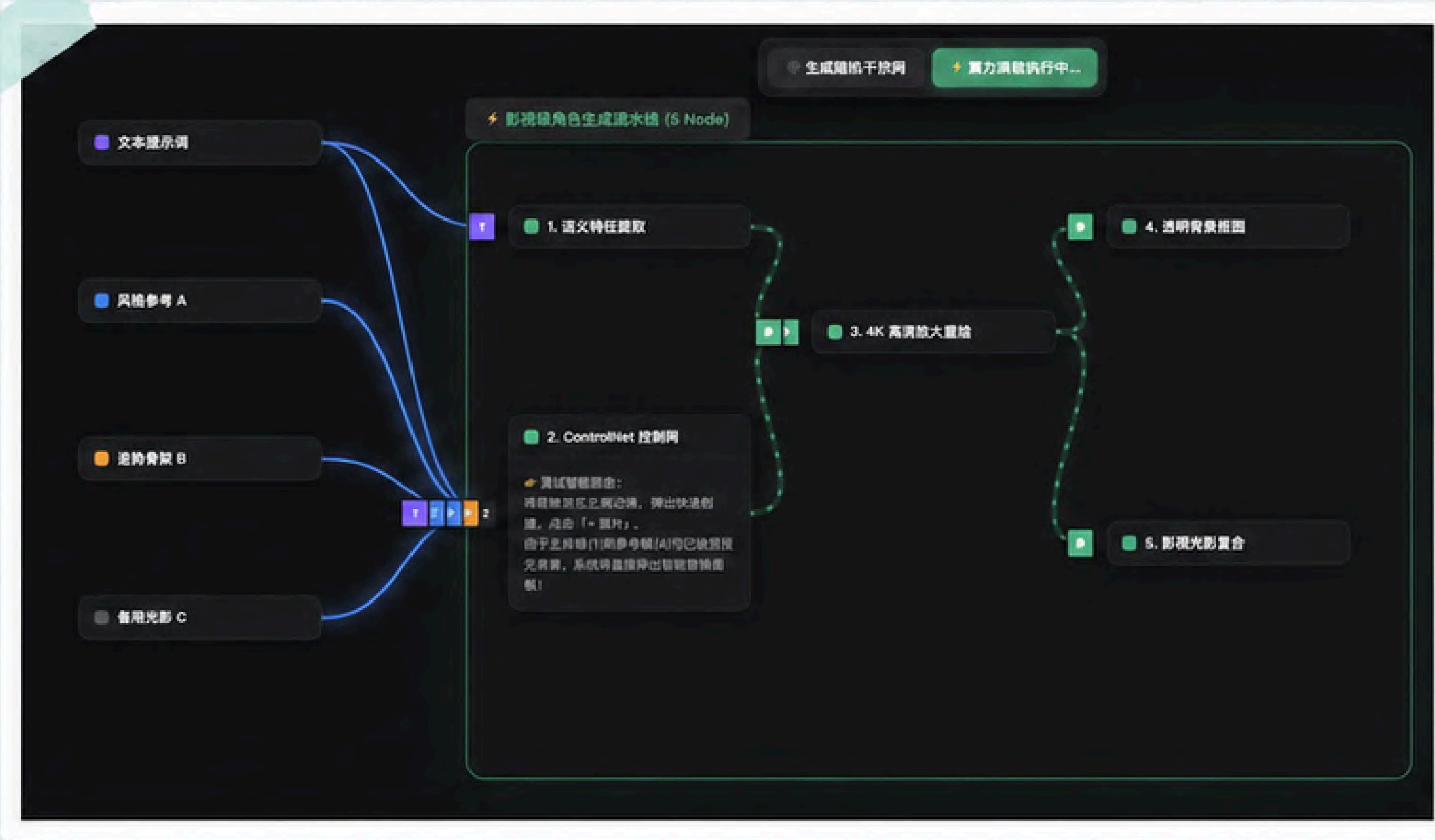
跑通行为

用 Demo 验证连线、等待、生成和编组是否可理解。
- 03

对齐交付

方向确认后，再沉淀高保真、交互规范与开发说明。

不是先画完再解释，而是先跑通再落地。



连线逻辑
验证节点编排
与数据流向。



生成反馈
验证等待、执行
与完成状态。



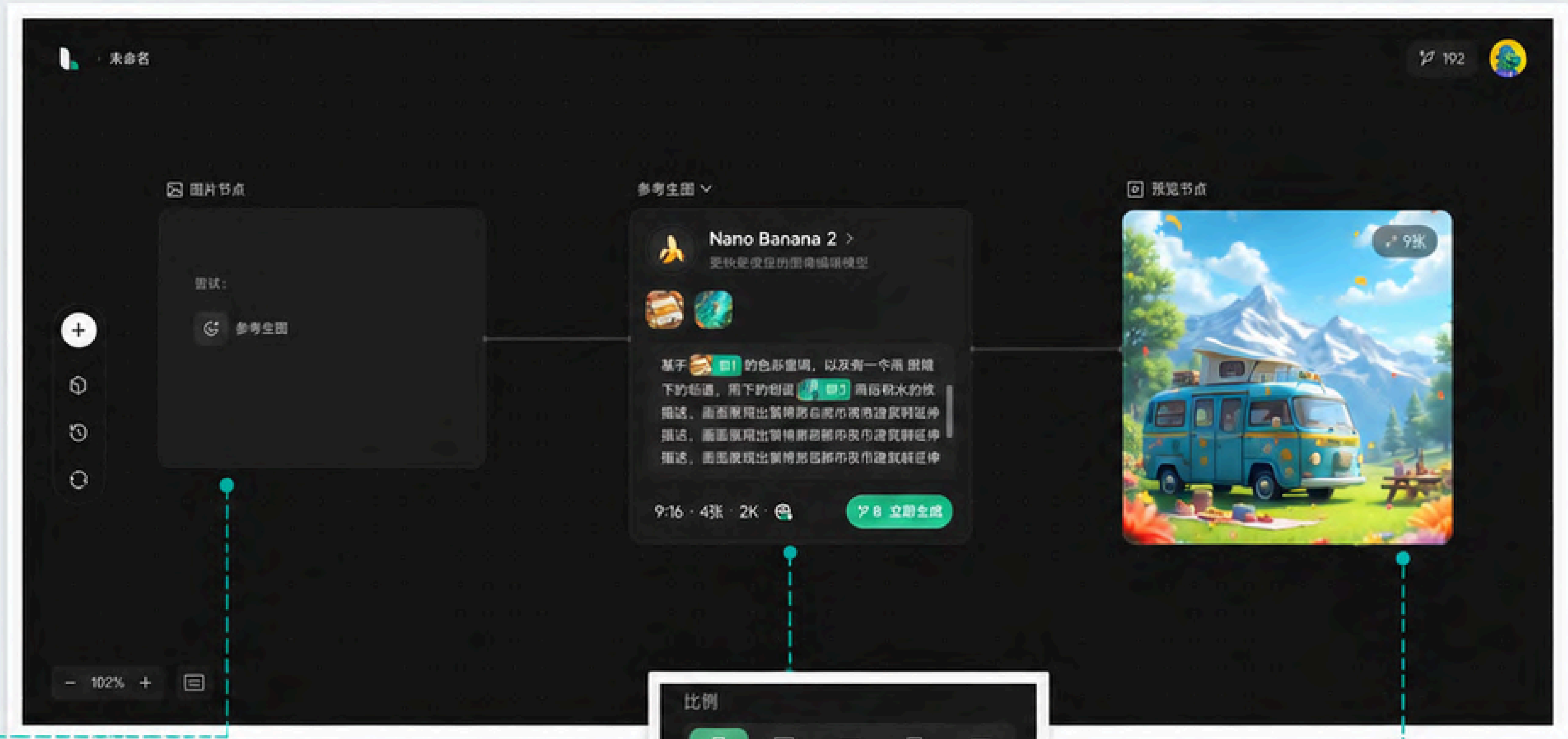
编组操作
验证选中、整组
执行与解组路径。

节点连接

CONNECTION

我先把输入、配置、结果拆开，再用连线保留上下文。

文本、图片或视频都可以作为输入；输出目标决定连接图片配置还是视频配置。参数确认后，结果节点继续承接输出。



01

输入有角色

文本、图片或视频先说明本次生成要使用什么。

02

配置定方向

连接图片或视频配置节点，在对应节点中确认模型与参数。



03

结果可承接

点击「立即生成」后，结果节点沿当前连接继续输出。

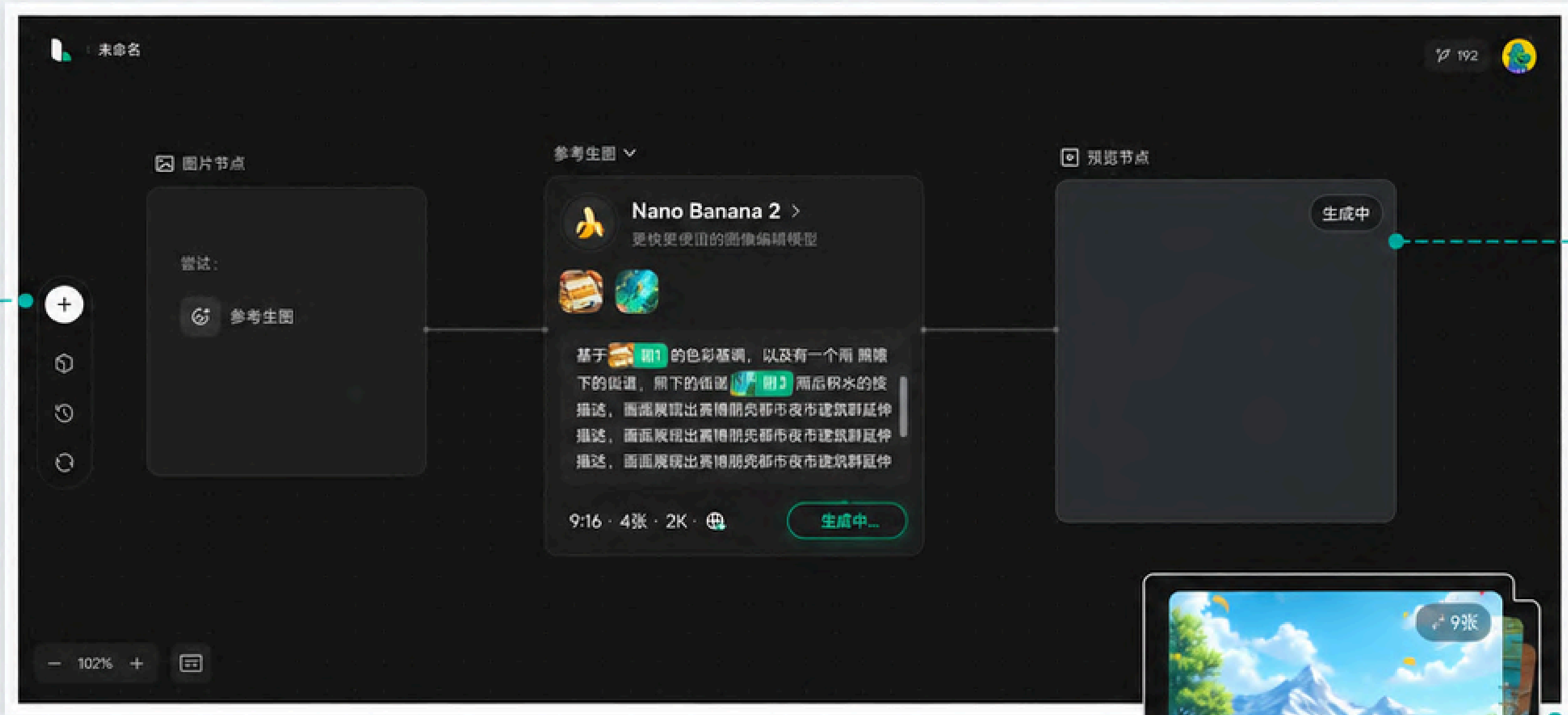
连接不是装饰，而是用户理解生成路径的线索。

生成反馈

FEEDBACK

我没有把生成做成黑箱，而是让等待和结果都有位置。
预览节点先承接「生成中」，完成后结果继续参与画布编排。

01 参数已确认
配置节点完成模型、
比例、画质与数量选择。



02 等待有位置
预览节点显示「生成中」，
提前占住结果落点。

03 结果可继续
完成后进入预览节点，
继续参与后续编排。

反馈不是额外提示，而是生成流程的一部分。

画布编组

CANVAS GROUPING

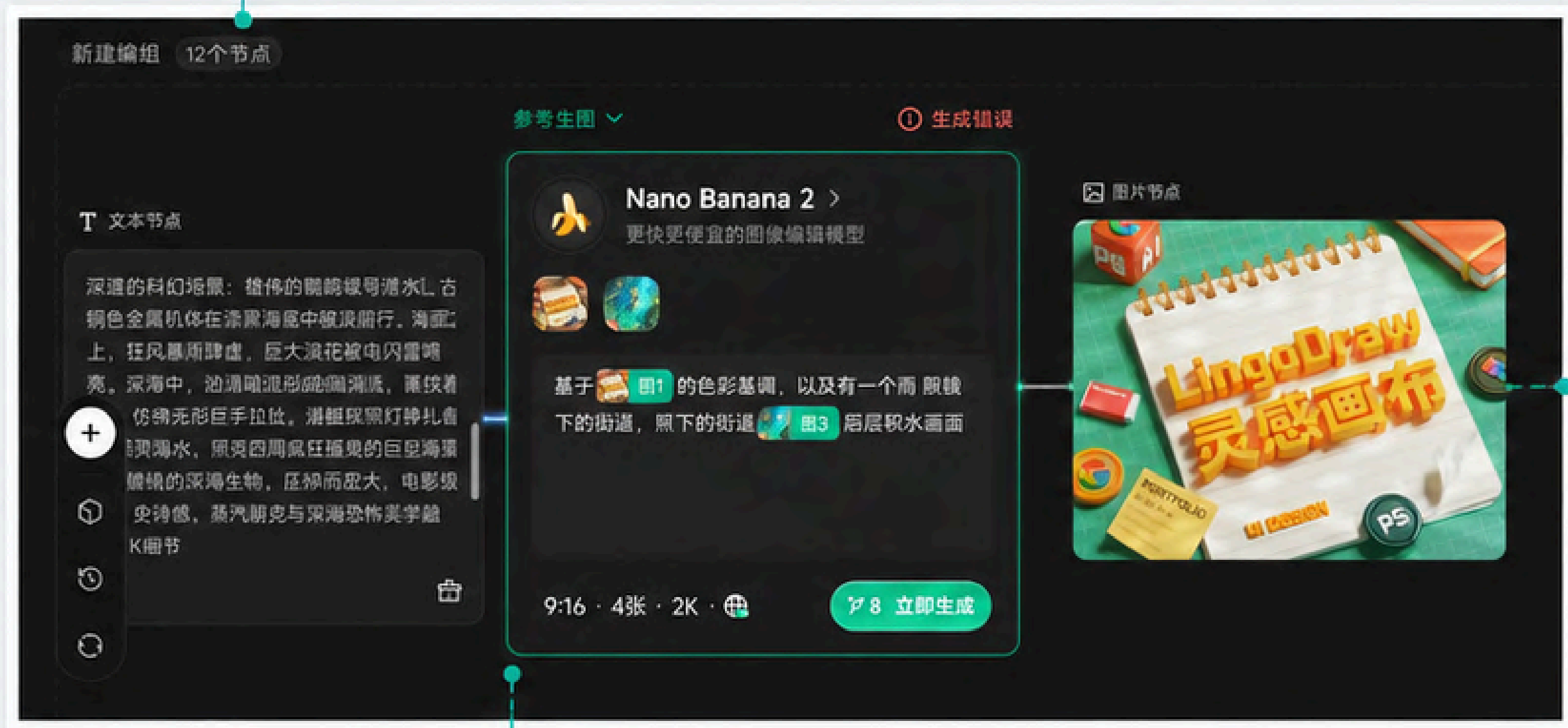
我没有把 **编组** 做成收纳，
而是让 **任务边界** 保持可读。

编组不是把节点收起来，
而是把 **2 个及以上** 相关节点
定义成 **同一段任务**。

编组前提：选中 2 个及以上节点。

单个节点仍保持节点级操作，
不进入编组状态。

01 成立条件
2 个及以上相关节点，
才进入编组。



02 边界可见
边界标出任务范围，
组内节点保持展开。

03 关系保留
跨边界连线继续可追踪，
避免上下文断裂。

关系不被打断，复杂画布才真正可管理。



控制跟随整组
整组执行、背景标识与解组，
都作用于同一组。

边界场景

EDGE CASES

遇到边界，不让用户猜；
当前状态与下一步就地说明。

从空画布起步、参数确认到生成等待，
反馈始终贴近当前操作。

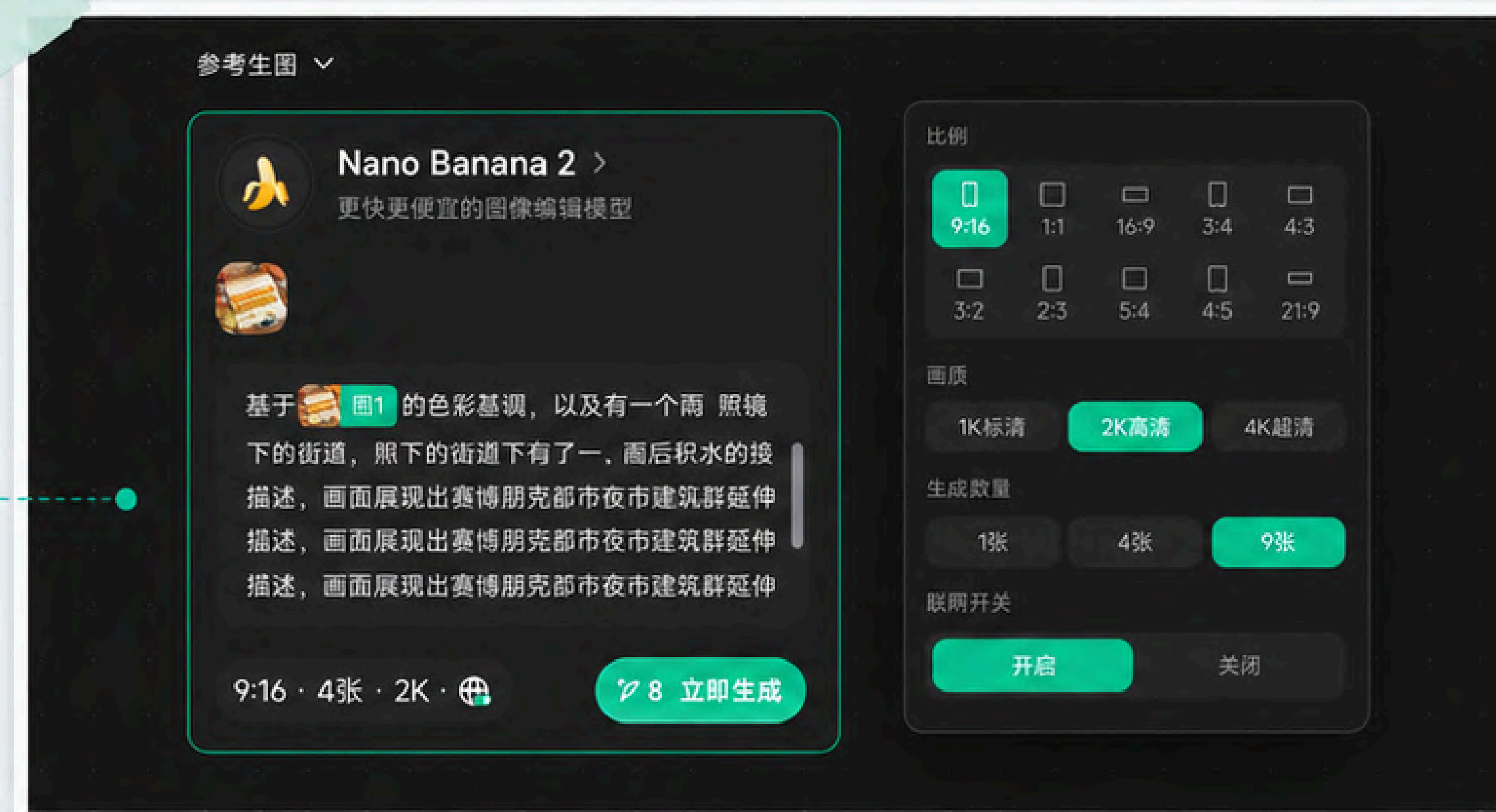
01 起步有入口

空画布直接提示创建方式，
不让用户面对无响应的空白。



02 选择有范围

比例、画质、数量与
联网选项在节点旁完成，
不打断当前链路。



03 等待有落点

预览节点提前出现并标注「生成中」，让用户知道结果会落在哪里。



设计系统

SYSTEM

我没有把设计系统做成组件合集，
而是把状态、语义和反馈沉淀成可复用规则。

节点怎么构成、按钮怎么变化、语义怎么反馈，
都能被开发按同一套语言还原。

01 结构有骨架

标题、内容区、端口与操作区保持一致。

02 状态成套定义

默认、悬停、选中、禁用和运行不各自为政。

03 语义能追踪

Tag 与状态反馈说明输入角色、关系和规则归属。



项目复盘

REFLECTION

这次项目里，我做得最重要的事，
是先把一条**复杂创作链路**跑通。

节点职责怎么拆、连接怎么解释、生成状态怎么反馈、
编组怎么保持上下文，我先用低保真和 Demo 验证，
再沉淀到高保真、设计系统和交互说明。

前面几页都在回答同一个问题：复杂创作怎么被一步步讲清楚。

复盘后，我更确定的事

01 先问关系

不要先堆功能，先问输入、配置、结果各自承担什么。

02 先跑行为

关键交互先用低保真和 Demo 跑通，
再进入高保真细化。

03 结果不是终点

生成结果不只是出图，
还要帮助用户比较、取舍，并继续回到画布创作。

下一轮，我会把重点放到“生成之后”：用户怎么比较、怎么留下、怎么继续用。

GENERATE, THEN DECIDE.

感谢阅览

T H A N K Y O U

谢谢看到这里。